

test_new_data

April 21, 2026

1 Test Trained Attack Detection Model on New PCAP Data

1.1 1. Title and Introduction

This notebook tests a trained machine learning model on a new network capture file for the attack-lab project.

The workflow converts `pcaps/new_test_capture.pcapng` into packet-level CSV data, preprocesses the same features used during training, loads the saved Random Forest model from `models/random_forest_attack_detection.joblib`, predicts traffic classes, saves prediction results, visualizes the predicted class distribution, and prints a final lab-style interpretation.

The model predicts three traffic classes:

- Normal
- Ping_Flood
- SYN_Flood

1.2 2. Import Libraries

Import libraries for file handling, PCAP conversion, data preprocessing, model loading, prediction, and visualization.

```
[1]: # Standard library imports
import os
import shutil
import subprocess
from pathlib import Path

# Keep Matplotlib from trying to write inside a non-writable home config
↪directory
os.environ.setdefault("MPLCONFIGDIR", "/tmp")

# Data and model libraries
import joblib
import pandas as pd

# Visualization libraries
import matplotlib
matplotlib.use("Agg")
```

```

import matplotlib.pyplot as plt

try:
    import seaborn as sns
except ModuleNotFoundError:
    sns = None

# Notebook display helper
try:
    from IPython.display import Markdown, display
except ImportError:
    Markdown = None
    display = print

```

1.3 3. Configure Paths and Fields

Define project paths, the new PCAP input, output CSV files, model path, and the packet fields extracted with tshark.

```

[2]: # Detect the attack-lab project root whether the notebook runs from project_
    ↪root or notebooks/
current_dir = Path.cwd().resolve()
if (current_dir / "pcaps").exists() and (current_dir / "models").exists():
    PROJECT_ROOT = current_dir
elif (current_dir.parent / "pcaps").exists() and (current_dir.parent /
    ↪"models").exists():
    PROJECT_ROOT = current_dir.parent
else:
    raise FileNotFoundError("Could not locate the attack-lab project root.")

PCAP_PATH = PROJECT_ROOT / "pcaps" / "new_test_capture.pcapng"
CSV_DIR = PROJECT_ROOT / "csv"
MODEL_PATH = PROJECT_ROOT / "models" / "random_forest_attack_detection.joblib"
RESULTS_DIR = PROJECT_ROOT / "results"

RAW_TEST_CSV = CSV_DIR / "new_test.csv"
PREDICTIONS_CSV = CSV_DIR / "new_test_predictions.csv"
PREDICTION_PLOT = RESULTS_DIR / "new_test_prediction_distribution.png"

TSHARK_FIELDS = [
    "frame.time_epoch",
    "ip.src",
    "ip.dst",
    "ip.proto",
    "frame.len",
    "tcp.srcport",
    "tcp.dstport",

```

```

        "tcp.flags.syn",
        "tcp.flags.ack",
        "tcp.flags.reset",
        "ip.ttl",
        "icmp.type",
        "icmp.code",
    ]

    NUMERIC_COLUMNS = [
        "frame.time_epoch",
        "ip.proto",
        "frame.len",
        "tcp.srcport",
        "tcp.dstport",
        "tcp.flags.syn",
        "tcp.flags.ack",
        "tcp.flags.reset",
        "ip.ttl",
        "icmp.type",
        "icmp.code",
    ]

    TRAINING_FEATURES = [
        "ip.proto",
        "frame.len",
        "tcp.srcport",
        "tcp.dstport",
        "tcp.flags.syn",
        "tcp.flags.ack",
        "tcp.flags.reset",
        "ip.ttl",
        "icmp.type",
        "icmp.code",
        "inter_arrival",
    ]

    DEFAULT_LABEL_MAP = {
        0: "Normal",
        1: "Ping_Flood",
        2: "SYN_Flood",
    }

    print("Project root:", PROJECT_ROOT)
    print("Input PCAP:", PCAP_PATH.relative_to(PROJECT_ROOT))
    print("Model path:", MODEL_PATH.relative_to(PROJECT_ROOT))

```

Project root: /home/durgaumadev/security-for-data-science-lab/attack-lab
 Input PCAP: pcaps/new_test_capture.pcapng

Model path: models/random_forest_attack_detection.joblib

1.4 4. Step 1: Convert PCAP to CSV

Use subprocess to run tshark and extract packet fields from pcaps/new_test_capture.pcapng into csv/new_test.csv.

The command uses the required format:

- -T fields
- -E header=y
- -E separator=,

```
[3]: def build_tshark_command(input_file: Path) -> list[str]:
    """Build the tshark command used to extract packet fields."""
    command = [
        "tshark",
        "-r",
        str(input_file),
        "-T",
        "fields",
        "-E",
        "header=y",
        "-E",
        "separator=",
        "-E",
        "occurrence=f",
    ]

    for field in TSHARK_FIELDS:
        command.extend(["-e", field])

    return command

def convert_pcap_to_csv() -> None:
    """Convert the new PCAPNG capture to csv/new_test.csv."""
    if shutil.which("tshark") is None:
        raise SystemExit(
            "tshark was not found on PATH. Install Wireshark/tshark before_␣
↳running this notebook."
        )

    if not PCAP_PATH.exists():
        raise FileNotFoundError(f"Missing PCAP file: {PCAP_PATH}")

    CSV_DIR.mkdir(parents=True, exist_ok=True)

    command = build_tshark_command(PCAP_PATH)
```

```

print("Running tshark command:")
print(" ".join(command))

with RAW_TEST_CSV.open("w", encoding="utf-8", newline="") as csv_handle:
    result = subprocess.run(
        command,
        stdout=csv_handle,
        stderr=subprocess.PIPE,
        text=True,
        check=False,
    )

    if result.returncode != 0:
        raise RuntimeError(
            f"tshark failed with exit code {result.returncode}:\n{result.stderr.
strip()}"
        )

    print(f"Converted PCAP to CSV: {RAW_TEST_CSV.relative_to(PROJECT_ROOT)}")
    print(f"CSV size: {RAW_TEST_CSV.stat().st_size / 1024:.1f} KB")

convert_pcap_to_csv()

```

Running tshark command:

```

tshark -r /home/durgaumadev/security-for-data-science-lab/attack-
lab/pcaps/new_test_capture.pcapng -T fields -E header=y -E separator=, -E
occurrence=f -e frame.time_epoch -e ip.src -e ip.dst -e ip.proto -e frame.len -e
tcp.srcport -e tcp.dstport -e tcp.flags.syn -e tcp.flags.ack -e tcp.flags.reset
-e ip.ttl -e icmp.type -e icmp.code
Converted PCAP to CSV: csv/new_test.csv
CSV size: 150.6 KB

```

1.5 5. Step 2: Load and Preprocess Data

Load `csv/new_test.csv`, fill missing values, convert numeric packet fields, sort packets by times-
tamp, create the `inter_arrival` feature, and select only the training features.

```

[4]: # Load the newly extracted CSV file
if not RAW_TEST_CSV.exists() or RAW_TEST_CSV.stat().st_size == 0:
    raise FileNotFoundError(f"Missing or empty CSV file: {RAW_TEST_CSV}")

new_df = pd.read_csv(RAW_TEST_CSV)

# Handle missing tshark columns safely by creating them with zeros
for column in TSHARK_FIELDS:
    if column not in new_df.columns:
        new_df[column] = 0

```

```

# Convert numeric columns properly
for column in NUMERIC_COLUMNS:
    new_df[column] = pd.to_numeric(new_df[column], errors="coerce")

# Fill missing metadata and numeric values
new_df["ip.src"] = new_df["ip.src"].fillna("0")
new_df["ip.dst"] = new_df["ip.dst"].fillna("0")
new_df = new_df.fillna(0)

# Sort by timestamp and create inter-arrival time
new_df = new_df.sort_values("frame.time_epoch").reset_index(drop=True)
new_df["inter_arrival"] = new_df["frame.time_epoch"].diff().fillna(0)

print("New test data shape:", new_df.shape)
print("Column names:")
print(new_df.columns.tolist())

new_df.head()

```

New test data shape: (2318, 14)

Column names:

```
['frame.time_epoch', 'ip.src', 'ip.dst', 'ip.proto', 'frame.len', 'tcp.srcport',
'tcp.dstport', 'tcp.flags.syn', 'tcp.flags.ack', 'tcp.flags.reset', 'ip.ttl',
'icmp.type', 'icmp.code', 'inter_arrival']
```

```

[4]:
   frame.time_epoch      ip.src      ip.dst  ip.proto  frame.len  \
0    1.776769e+09  10.10.166.245  10.10.166.129      1.0      98
1    1.776769e+09  10.10.166.129  10.10.166.245      1.0      98
2    1.776769e+09  10.10.166.245  10.10.166.129      1.0      98
3    1.776769e+09  10.10.166.129  10.10.166.245      1.0      98
4    1.776769e+09  10.10.166.245  10.10.166.129      1.0      98

   tcp.srcport  tcp.dstport  tcp.flags.syn  tcp.flags.ack  tcp.flags.reset  \
0           0.0           0.0           0.0           0.0           0.0
1           0.0           0.0           0.0           0.0           0.0
2           0.0           0.0           0.0           0.0           0.0
3           0.0           0.0           0.0           0.0           0.0
4           0.0           0.0           0.0           0.0           0.0

   ip.ttl  icmp.type  icmp.code  inter_arrival
0    64.0        8.0        0.0      0.000000
1   128.0         0.0        0.0      0.000689
2    64.0        8.0        0.0      0.018142
3   128.0         0.0        0.0      0.000678
4    64.0        8.0        0.0      0.019386

```

1.6 6. Step 3: Load Trained Model

Load `models/random_forest_attack_detection.joblib`. The saved file is a dictionary containing the trained model, feature columns, and label map.

```
[5]: # Load the saved Random Forest joblib artifact
if not MODEL_PATH.exists():
    raise FileNotFoundError(f"Missing trained model file: {MODEL_PATH}")

model_artifact = joblib.load(MODEL_PATH)
model = model_artifact["model"]
feature_columns = model_artifact.get("feature_columns", TRAINING_FEATURES)
label_map = model_artifact.get("label_map", DEFAULT_LABEL_MAP)

# Normalize label map keys defensively
label_map = {int(key): value for key, value in label_map.items()}

print("Loaded model artifact:", MODEL_PATH.relative_to(PROJECT_ROOT))
print("Model type:", type(model).__name__)
print("Feature columns from artifact:")
print(feature_columns)
print("Label map:", label_map)
```

```
Loaded model artifact: models/random_forest_attack_detection.joblib
Model type: RandomForestClassifier
Feature columns from artifact:
['ip.proto', 'frame.len', 'tcp.srcport', 'tcp.dstport', 'tcp.flags.syn',
'tcp.flags.ack', 'tcp.flags.reset', 'ip.ttl', 'icmp.type', 'icmp.code',
'inter_arrival']
Label map: {0: 'Normal', 1: 'Ping_Flood', 2: 'SYN_Flood'}
```

1.7 7. Select Model Features

Select the same features used during training. Any missing feature is created with zero so prediction does not fail on captures that do not contain every protocol field.

```
[6]: # Ensure every training feature exists before prediction
for column in feature_columns:
    if column not in new_df.columns:
        new_df[column] = 0

# Select the model input features in the exact training order
X_new = new_df[feature_columns].copy()
X_new = X_new.apply(pd.to_numeric, errors="coerce").fillna(0)

print("Prediction feature matrix shape:", X_new.shape)
X_new.head()
```

```
Prediction feature matrix shape: (2318, 11)
```

```
[6]:   ip.proto  frame.len  tcp.srcport  tcp.dstport  tcp.flags.syn  \
0      1.0      98      0.0      0.0      0.0
1      1.0      98      0.0      0.0      0.0
2      1.0      98      0.0      0.0      0.0
3      1.0      98      0.0      0.0      0.0
4      1.0      98      0.0      0.0      0.0

      tcp.flags.ack  tcp.flags.reset  ip.ttl  icmp.type  icmp.code  inter_arrival
0           0.0           0.0   64.0      8.0      0.0      0.000000
1           0.0           0.0  128.0      0.0      0.0      0.000689
2           0.0           0.0   64.0      8.0      0.0      0.018142
3           0.0           0.0  128.0      0.0      0.0      0.000678
4           0.0           0.0   64.0      8.0      0.0      0.019386
```

1.8 8. Step 4: Predict Traffic Classes

Use `model.predict()` to classify packets and convert numeric predictions into readable labels:

- 0 = Normal
- 1 = Ping_Flood
- 2 = SYN_Flood

```
[7]: # Predict numeric labels and map them to readable class names
predictions = model.predict(X_new)
predicted_labels = pd.Series(predictions).astype(int).map(label_map).
    ↪ fillna("Unknown")

# Add predictions to a copy of the preprocessed dataframe
prediction_df = new_df.copy()
prediction_df["predicted_label"] = predicted_labels

print("Prediction counts:")
prediction_counts = prediction_df["predicted_label"].value_counts()
print(prediction_counts)

prediction_df.head()
```

```
Prediction counts:
predicted_label
Ping_Flood      1991
Normal           327
Name: count, dtype: int64
```

```
[7]:   frame.time_epoch      ip.src      ip.dst  ip.proto  frame.len  \
0  1.776769e+09  10.10.166.245  10.10.166.129      1.0      98
1  1.776769e+09  10.10.166.129  10.10.166.245      1.0      98
2  1.776769e+09  10.10.166.245  10.10.166.129      1.0      98
3  1.776769e+09  10.10.166.129  10.10.166.245      1.0      98
```


4	1.776769e+09	10.10.166.245	10.10.166.129	1.0	98
---	--------------	---------------	---------------	-----	----

	tcp.srcport	tcp.dstport	tcp.flags.syn	tcp.flags.ack	tcp.flags.reset \
0	0.0	0.0	0.0	0.0	0.0
1	0.0	0.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0
4	0.0	0.0	0.0	0.0	0.0

	ip.ttl	icmp.type	icmp.code	inter_arrival	predicted_label
0	64.0	8.0	0.0	0.000000	Normal
1	128.0	0.0	0.0	0.000689	Ping_Flood
2	64.0	8.0	0.0	0.018142	Ping_Flood
3	128.0	0.0	0.0	0.000678	Ping_Flood
4	64.0	8.0	0.0	0.019386	Ping_Flood

1.9 9. Step 5: Save Prediction Results

Save the packet-level prediction output to `csv/new_test_predictions.csv`.

```
[8]: # Save packet-level predictions to CSV
prediction_df.to_csv(PREDICTIONS_CSV, index=False)

print(f"Saved predictions to: {PREDICTIONS_CSV.relative_to(PROJECT_ROOT)}")
print(f"Prediction CSV size: {PREDICTIONS_CSV.stat().st_size / 1024:.1f} KB")
```

Saved predictions to: `csv/new_test_predictions.csv`

Prediction CSV size: 269.4 KB

1.10 10. Step 6: Visualization

Plot the predicted class distribution using seaborn when available. The plot is also saved to `results/new_test_prediction_distribution.png`.

```
[9]: # Plot and save predicted class distribution
RESULTS_DIR.mkdir(parents=True, exist_ok=True)

class_order = ["Normal", "Ping_Flood", "SYN_Flood"]
counts = prediction_df["predicted_label"].value_counts().reindex(class_order,
    ↪fill_value=0)

plt.figure(figsize=(8, 5))
if sns is not None:
    sns.countplot(
        data=prediction_df,
        x="predicted_label",
        hue="predicted_label",
        order=class_order,
```

```

        palette="Set2",
        legend=False,
    )
else:
    print("Warning: seaborn is not installed; using matplotlib for the plot.")
    plt.bar(counts.index, counts.values, color=["#66c2a5", "#fc8d62",
↪ "#8da0cb"])

plt.title("New Test Capture Prediction Distribution")
plt.xlabel("Predicted Traffic Class")
plt.ylabel("Packet Count")
plt.tight_layout()
plt.savefig(PREDICTION_PLOT, dpi=150)
plt.show()

print(f"Saved plot to: {PREDICTION_PLOT.relative_to(PROJECT_ROOT)}")

```

Saved plot to: results/new_test_prediction_distribution.png

/tmp/ipykernel_131149/1483341328.py:26: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown

```
plt.show()
```

1.11 11. Step 7: Final Interpretation

Identify the majority predicted class and print a short summary of the new capture.

```

[10]: # Print a lab-style summary based on the majority prediction
if prediction_df.empty:
    majority_label = "Unknown"
    interpretation = "No packets were available for prediction."
else:
    majority_label = prediction_df["predicted_label"].value_counts().idxmax()
    interpretation = (
        "The traffic is predominantly "
        f"{majority_label} based on majority prediction"
    )

print(interpretation)

if Markdown is not None:
    display(Markdown(f"**Final Interpretation:** {interpretation}"))

```

The traffic is predominantly Ping_Flood based on majority prediction

Final Interpretation: The traffic is predominantly Ping_Flood based on majority prediction

1.12 Result

This notebook converts the new PCAP capture into packet-level features, loads the trained Random Forest attack detection model, predicts labels for each packet, saves the prediction CSV, visualizes the class distribution, and reports the majority traffic class.

The generated outputs are:

- `csv/new_test.csv`
- `csv/new_test_predictions.csv`
- `results/new_test_prediction_distribution.png`